

Cover Page

Course Title: **STUDY PROJECT (BCS ZC241T)**

Project Title: *Smart Workflow-Based Queue and Administrative Service Management System for Educational Institutions*

GROUP Number: **133**

Student Name(s):

- 1. BHUPESH KUMAR**
- 2. GONE MANJUSRI**
- 3. HARSH UPADHYAY**
- 4. HARANGAD SINGH GHAI**

Student ID(s):

- 1. 2023ebcs402**
- 2. 2023ebcs404**
- 3. 2023ebcs445**
- 4. 2023ebcs526**

Project Advisor / Supervisor: **DHRUVIKA CHAUHAN**

Date of Submission:

PHASE 2 *Smart Workflow-Based Queue and Administrative Service Management System for Educational Institutions*

1 Introduction

1.1 Purpose of Phase 2

Phase 2 turns the Phase-1 idea into a real technical plan.

Its main goals are:

- Convert the Phase-1 problem statement into a concrete system design (modules, data model, workflows).
- Define clear functional and non-functional requirements that developers can follow.
- Create the architecture and detailed diagrams needed for implementation.
- Build a small Proof of Concept (PoC) to show the design works and to catch risks early.
- Prepare everything needed to move from design to actual coding (APIs, DB schema, worker/jobs design, caching strategy).

1.2 Scope of Phase 2

Phase 2 includes the following deliverables and work items:

- System design and architecture diagrams (HLD and module view).
- Detailed functional requirements and non-functional requirements derived from Phase-1.
- Complete database design: ER diagram, collections, keys, and important attributes.
- Class-level LLD (UML class diagram), workflow models, and sequence/flow diagrams.
- API design (endpoints, request/response shapes, auth rules).
- Queue & appointment design, Redis key patterns, and BullMQ job definitions.
- Document storage integration plan (ImageKit metadata flow).
- Chatbot design overview (deterministic + RAG usage and knowledge index).
- Background worker design and the rule that workers call service layer functions (not DB models).
- A working PoC that demonstrates core flows (submit request, workflow step, document upload, queue/appointment, and one chatbot query).
- Test checklist and acceptance criteria for the PoC.

2 System Overview

2.1 Product Perspective

- The system is a standalone web-based college service platform. It works independently and does not replace existing ERP systems. Data can be exported for use in other systems when required.
- Users access the system through web applications:
 - Student Portal
 - Staff Dashboard
 - Admin Console
- All client applications communicate with a monolithic backend system built using Node.js and Express.
- The backend follows a modular monolithic architecture, where all modules (authentication, workflow, requests, queue, chatbot, etc.) are part of a single deployable application.
- Stateful authentication is used:
 - User login sessions are maintained on the server.
 - Session information is stored and validated on every request.
- The backend is responsible for:
 - Request validation and authorization
 - Workflow execution
 - Queue and appointment handling
 - Document management
 - Chatbot processing
- The system uses:
 - MongoDB as the primary database
 - Redis for caching, real-time queue state, and background job handling(BULLMQ)
 - ImageKit for document and image storage
 - A knowledge index / vector database for chatbot RAG responses
- Deployment targets:
 - The application will be deployed as a web application.

- Hosting may be done on platforms like Vercel or a VPS in the future.
- The final deployment environment will be decided in later phases based on requirements.

2.2 Major System Functions

- Stateful user authentication and authorization
Server-side session-based login with role-based access control for students, staff, and admins.
- Service catalog and offering management
Admins can define services and control their availability through service offerings.
- Service request creation and tracking
Students submit service requests and track progress through workflow stages.
- Workflow-driven request processing
Each service request is processed using a predefined workflow executed step-by-step.
- Document upload and verification
Students upload documents, and staff verify them during workflow processing.
- Hybrid queue and appointment handling
The system supports virtual queues for walk-in services and appointment scheduling for complex cases.
- Chatbot support (Deterministic + RAG)
The chatbot assists users with FAQs, eligibility checks, and guided instructions.
- Background job execution
Asynchronous tasks such as scheduling, escalation, and reminders are handled by background workers.
- Data export functionality
Service and request data can be exported in CSV format for reporting or external use.

2.3 User Classes and Characteristics

Students (End Users)

Students are the primary users of the system. They use the student portal to:

- View available services and service requirements
- Submit service requests
- Upload required documents
- Join queues or book appointments

- Track request status and workflow progress
- Use the chatbot for help and guidance

Students typically have basic technical knowledge and access the system through a web browser.

Staff

Staff members are responsible for processing service requests. They use the staff dashboard to:

- View assigned service requests
- Verify and approve uploaded documents
- Process workflow steps in sequence
- Update request status and add remarks

Staff users have role-based access limited to services and workflow steps assigned to them.

Administrators

Administrators manage and control the system configuration. They use the admin console to:

- Create and manage services and service offerings
- Define workflow templates and steps
- Manage user roles and permissions
- Monitor system usage and export data

Administrators have full access to system configuration features.

External Systems / APIs

External systems interact with the platform indirectly. These include:

- Document storage services (for file uploads and retrieval)
- Chatbot knowledge services (for RAG-based responses)
- Exported data consumers (CSV files used by ERP or reporting tools)

These systems do not directly access the user interface and communicate only through secure APIs or data exports.

3. Functional Requirements

Functional requirements describe what the system must do to support college administrative services. The system is designed as a stateful monolithic application with authentication and Redis-based caching.

FR1: User Authentication and Stateful Session Management

- The system shall allow students, staff, and administrators to authenticate.
- User sessions shall be stored and managed on the server.
- Users shall remain logged in until they log out or the session expires.

FR2: Role-Based Access Control

- The system shall provide role-based access to features.
- Students can create and track service requests.
- Staff can process and verify service requests.
- Admin can manage services, workflows, and system settings.

FR3: Service Management

- The system shall allow administrators to create and manage college services.
- Each service represents a specific administrative process.
- Services can be enabled or disabled as required.

FR4: Workflow Management

- The system shall allow administrators to define workflows for each service.
- A workflow consists of multiple ordered steps.
- Each workflow step is assigned to a specific role.
- Workflow steps must be completed sequentially.

FR5: Service Request Creation

- The system shall allow students to create service requests.
- Each request shall be linked to a selected service and its workflow.
- A unique request ID shall be generated.
- The request shall start from the first workflow step.

FR6: Document Upload and Management

- The system shall allow students to upload required documents.
- Documents shall be stored securely.
- Each document shall have a status: pending, verified, or rejected.

FR7: Virtual Queue Management

- The system shall support a virtual queue for applicable services.
- Students shall be assigned a queue position.
- Queue positions shall update dynamically as requests are processed.
- Queue data may be cached for faster access.

FR8: Appointment Booking System

- The system shall allow students to book appointments for services requiring physical presence.
- Appointment slots shall be predefined by the system.
- The system shall prevent double booking of slots.

FR9: Staff Request Processing

- The system shall allow staff members to process service requests.
- Staff can claim requests for processing.
- Staff can verify documents.
- Staff can approve or reject workflow steps.

FR10: Request Status Tracking

- The system shall allow students to track request progress.
- The current workflow step shall be displayed.
- Queue position or appointment details shall be visible.

FR11: Chatbot Assistance System

- The system shall provide a chatbot to assist users.
- The chatbot shall answer service-related queries.
- The chatbot shall guide users about required documents and workflow steps.
- Chat history shall be maintained during the user session.

FR12: Knowledge-Based Chat Responses

- The chatbot shall provide responses based on stored service and workflow data.
- Deterministic answers shall be given for common queries.
- Stored knowledge shall support chatbot responses.

FR13: Redis-Based Caching

- The system shall use Redis to improve performance.
- Frequently accessed data such as service lists, request status, and queue positions shall be cached.
- Cache shall be updated when underlying data changes.

FR14: Administrative Configuration

- The system shall allow administrators to configure:
 - Services and workflows
 - Queue rules
 - Appointment rules
 - Basic system settings

FR15: Logout and Session Termination

- The system shall allow users to log out securely.
- Session data shall be cleared after logout.
- Inactive sessions shall expire automatically.

4. Non-Functional Requirements

4.1 Performance Requirements

NFR-P1: Response Time

- The system shall respond within 1–2 seconds for normal requests.
- Cached responses shall respond within 500 milliseconds.

NFR-P2: Load Handling

- The system shall support 50–100 concurrent users.
- Performance shall remain stable during peak usage.

NFR-P3: Cache Efficiency

- Redis caching shall reduce repeated database queries.
- Cache expiration shall ensure data consistency.

4.2 Security Requirements

NFR-S1: Authentication and Authorization

- User authentication shall be mandatory.
- Role-based authorization shall be enforced.

NFR-S2: Session Security

- Sessions shall expire after a defined inactivity period.
- Unauthorized access attempts shall be prevented.

NFR-S3: Data Privacy and Protection

- User data and documents shall be accessible only to authorized users.
- Sensitive information shall be protected from unauthorized access.

4.3 Usability Requirements

NFR-U1: Ease of Navigation

- The system shall have a simple and intuitive user interface.
- Core actions shall be completed with minimal steps.

NFR-U2: User Feedback

- The system shall provide clear success and error messages.
- Request status and errors shall be clearly displayed.

NFR-U3: Accessibility

- The system shall be usable on common devices and browsers.
- Text and layout shall be readable and clear.

4.4 Scalability and Maintainability

NFR-M1: Monolithic Maintainability

- The system shall follow a modular structure within a monolithic architecture.
- Each module shall be independently maintainable.

NFR-M2: Ease of Future Extension

- New services and workflows shall be added without major code changes.
- Redis caching rules shall be extendable.

NFR-M3: Code Quality and Documentation

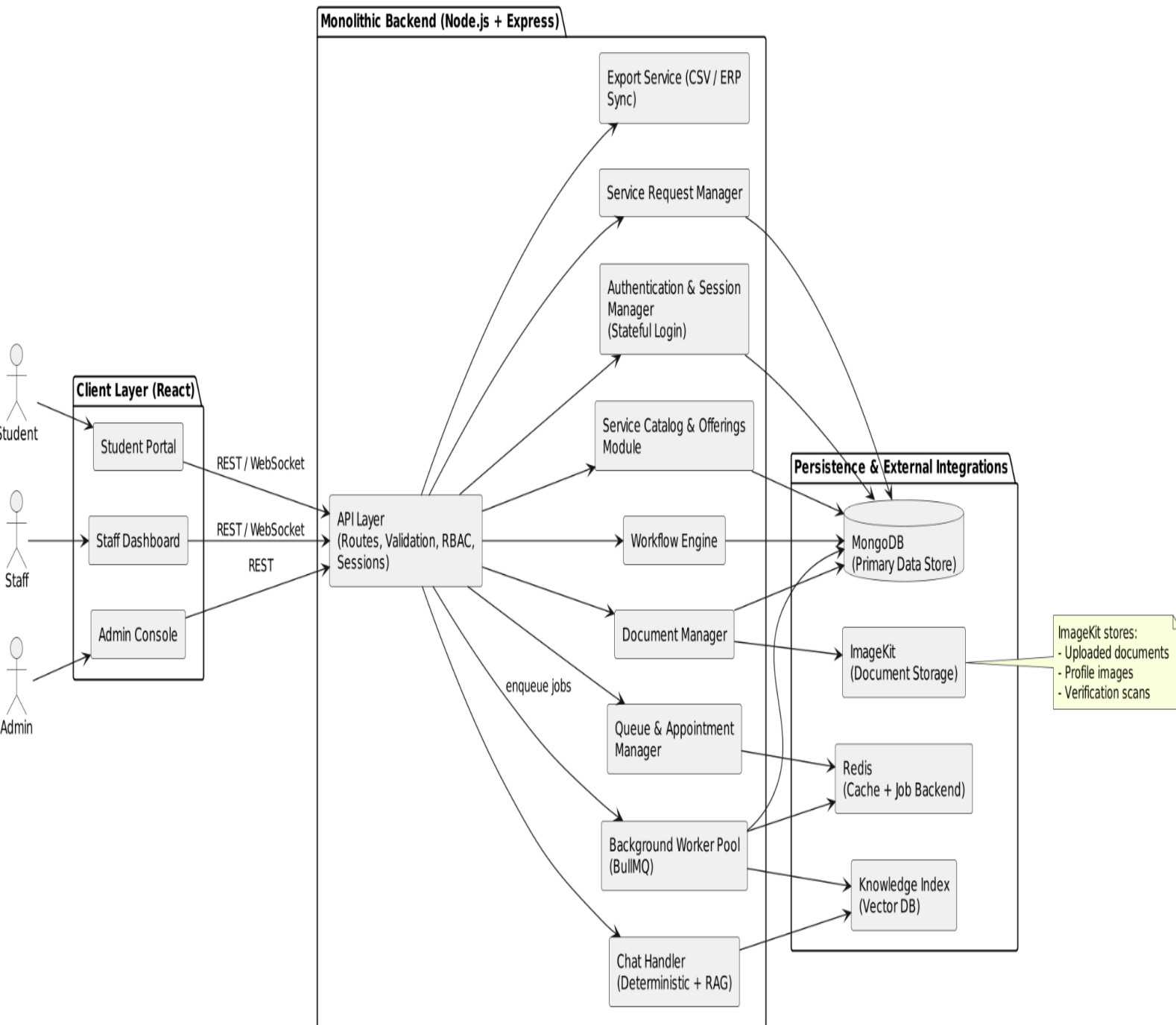
- The codebase shall be readable and well-structured.

- Basic documentation shall be provided for setup and maintenance.

5. System Architecture and Design

5.1 System Architecture Diagram

System Architecture Diagram - Smart Workflow-Based Queue and Administrative Service Management System for Educational Institutions



High-level architecture diagram

Architecture Explanation

Client Layer

The client layer consists of **three role-based React applications**:

- **Student Portal** – service discovery, chatbot assistance, queue/appointment booking, and request tracking

- **Staff Dashboard** – workflow step processing, document verification, and approvals
- **Admin Console** – service configuration, workflow setup, and system monitoring

All clients communicate with the backend using **REST APIs** and **WebSockets** for real-time updates.

Monolithic Backend

The backend is implemented as a **single Node.js + Express application**, structured modularly.

Key components include:

- **API Layer** – request routing, validation, role-based access control, and session handling
- **Authentication & Session Manager** – maintains stateful user sessions
- **Workflow Engine** – executes service workflows step-by-step
- **Service Request Manager** – manages request lifecycle and status tracking
- **Queue & Appointment Manager** – handles hybrid access using virtual queues and scheduled appointments
- **Document Manager** – manages metadata and integrates with ImageKit
- **Chat Handler** – processes chatbot queries using deterministic rules and RAG
- **Background Worker Pool** – executes asynchronous tasks (queue scheduling, escalation, notifications)
- **Export Service** – generates CSV outputs for ERP or record integration

Persistence Layer

- **MongoDB** is used as the primary datastore for users, services, workflows, requests, documents, and chat history
- **Redis** is used for:
 - Caching frequently accessed data
 - Queue state management
 - BullMQ job backend

External Services & APIs

- **ImageKit** is used for secure document and image storage
- **Vector Knowledge Index** supports Retrieval-Augmented Generation (RAG) for the chatbot

5.2 Module-wise Design

1. API Layer (Routes, Validation, RBAC, Sessions)

Responsibilities

- Expose REST endpoints and WebSocket channels used by all UIs (Student, Staff, Admin).
- Validate requests, enforce role-based access, and manage server-side sessions (stateful auth).

Inputs / Outputs

- Input: HTTP/WS requests from clients (JSON payloads, auth cookies/sessions).
- Output: JSON responses, WebSocket events, HTTP error codes, and enqueued background jobs.

Interactions

- Calls Auth/Session Manager for login/logout and session checks.
- Forwards validated requests to Service Request Manager, Catalog, Workflow, Queue/Appointment, Document Manager, Chat Handler, or Export Service.
- Enqueues background jobs in the Worker queue when needed.

2. Authentication & Session Manager (Stateful Auth)

Responsibilities

- Authenticate users and create/maintain server-side sessions.
- Enforce session expiry and logout behavior.

Inputs / Outputs

- Input: Login credentials, logout requests, session refresh checks.
- Output: Session cookie/token, session records, authentication errors.

Interactions

- Used by the API Layer on every protected request.
- Reads/Writes session info to a server session store (in-memory/DB/Redis as configured).
- Works with RBAC rules to decide access to Catalog, Workflow, Requests, and Admin functions.

3. Service Catalog & Offerings Module

Responsibilities

- Manage master list of services and their offerings (academic year, program, deadlines).
- Return service metadata used by UI and chatbot.

Inputs / Outputs

- Input: Admin requests to create/update services and offerings.
- Output: Service lists, offering details, document checklists, eligibility rules.

Interactions

- Reads/writes **services** and **service_offerings** collections in MongoDB.
- Provides data to the API Layer and Chat Handler (for RAG/deterministic responses).

4. Service Request Manager

Responsibilities

- Create and maintain runtime request records when a student submits a service request.
- Maintain **requestNumber**, status, current workflow step, and links to documents, queue/appointment.

Inputs / Outputs

- Input: New request payloads (userId, serviceId, offeringId, uploaded documents metadata).
- Output: Created **service_requests** record, acknowledgement to client, and initial workflow instance.

Interactions

- Writes to **service_requests** and triggers Workflow Engine to create a WorkflowInstance.
- If the user choose queue or appointment, calls Queue & Appointment Manager.
- Notifies API Layer / UI of status updates (via WebSocket or polling).

5. Workflow Engine (Template + Runtime)

Responsibilities

- Maintain workflow templates and execute workflow instances step by step. Templates define steps, assigned roles, SLA, and order.
- Track step status (pending/approved/rejected), process claims, and move the request forward.

Inputs / Outputs

- Input: **workflowTemplateId** for a request, staff actions (approve/reject), document verification results.
- Output: Updated WorkflowInstance and WorkflowStepInstance records; request status changes.

Interactions

- Reads **workflow_templates** and writes runtime **workflow_instance** + **workflow_step_instance** (stored in Mongo).
db-design
- Notifies Service Request Manager and API Layer when steps complete.
- Enqueues escalation or reminder jobs in the Worker queue for SLA deadlines.

6. Queue & Appointment Manager (Virtual Queue + Scheduling)

Responsibilities

- Manage virtual queue tokens and appointment slots. Calculate ETA, assign positions, prevent double booking.

Inputs / Outputs

- Input: Join-queue requests, appointment bookings, service capacity updates.
- Output: QueueEntry records, appointment records, ETA/position updates.

Interactions

- Writes **queues** and **appointments** collections in Mongo and stores real-time state in Redis for fast reads.
- Uses BullMQ workers for token scheduling, reordering, and reminders.
- Provides current queue/appointment info to Service Request Manager and the UI.

7. Document Manager (Image storage + metadata)

Responsibilities

- Accept and validate file uploads (documents, photos). Store metadata in Mongo (**documents** collection) and files in ImageKit or chosen external storage.

Inputs / Outputs

- Input: File uploads from clients (multipart/form-data).
- Output: **documents** records with **fileUrl**, upload status, and verification timestamps.

Interactions

- Calls external ImageKit for actual file storage; stores only URLs and metadata in Mongo.
Study-project Phase1
- Staff actions on documents feed back to Workflow Engine and Service Request Manager (verified/rejected).
- Document status cached briefly for fast UI feedback, invalidated on updates.

8. Chat Handler (Deterministic + RAG with Knowledge Index)

Responsibilities

- Handle chatbot conversations: deterministic answers for simple FAQs and RAG flow (retrieve knowledge documents + generate contextual answers via Gemini or configured model).

Inputs / Outputs

- Input: User chat messages, intent/context metadata.
- Output: Chat responses, suggested actions (e.g., "join queue", "upload document"), and chat session/activity logs.

Interactions

- Reads **knowledge_documents** and **chat_sessions** / **chat_messages** collections in Mongo.

- Calls the knowledge index (vector DB) for retrieval, then the external model API for generation.
- Can direct users to Service Catalog, start a request, or route them to join a queue/appointment.

9. Background Worker Pool (BullMQ + Redis)

Responsibilities

- Execute asynchronous jobs: token scheduling, deadline escalation, appointment reminders, CSV export jobs, retries. Workers pick jobs from Redis-backed queues.

Inputs / Outputs

- Input: Jobs enqueued by API or internal modules (job payloads).
- Output: Job results (e.g., updated queue positions, notifications sent, CSV file generated).

Interactions

- Reads and writes to Mongo and Redis via service-layer functions. **Workers should call service-layer functions instead of directly calling DB models.**
- Notifies API Layer or Upload/Exporter when jobs finish.

10. Database & Cache Layer (MongoDB + Redis + Knowledge Index)

Responsibilities

- MongoDB: persistent store for users, services, offerings, workflow templates, requests, documents, chat sessions/messages, queues/appointments.
- Redis: cache for read-heavy items (service list, request summaries) and stores real-time queue state + BullMQ job backend.
- Knowledge Index (vector DB): stores embeddings / index used by the chatbot RAG flow.

Inputs / Outputs

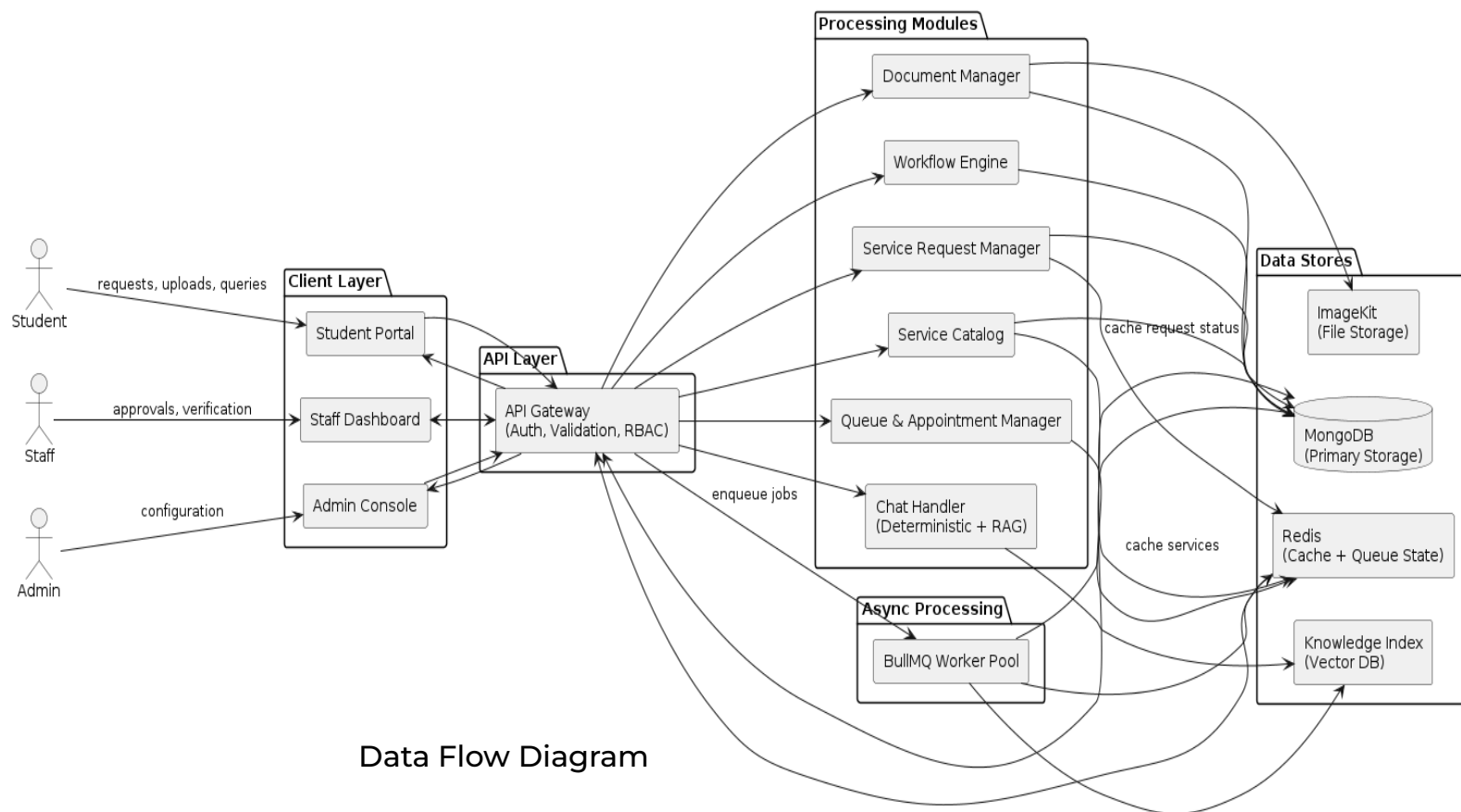
- Input: CRUD operations from modules.
- Output: Cached responses, persisted records, retrievals for chat and UI.

Interactions

- All modules read/write through the service layer which handles transaction boundaries, caching rules, and invalidation.

5.3 Data Flow Design

This section explains how data moves through the system from input to processing and storage. The system follows a **workflow-based data flow** using a **monolithic backend, MongoDB, Redis, and background workers**.



Data Flow Diagram

Input Sources

- **Students** use the web portal to log in, view services, submit requests, upload documents, join queues, and use the chatbot.
- **Staff** use the dashboard to verify documents and process workflow steps.
- **Admins** configure services, offerings, and workflows through the admin panel.

All inputs are sent to the backend through **REST APIs** or **WebSockets**.

Processing Stages

1. **Request Validation**
Requests first reach the API layer where user sessions and roles are checked.
2. **Business Logic Processing**
Valid requests are handled by modules such as:
 - Service Request Manager
 - Workflow Engine
 - Queue & Appointment Manager
 - Document Manager
 - Chat Handler
3. **Asynchronous Processing**
Tasks like queue updates, reminders, and exports are processed by

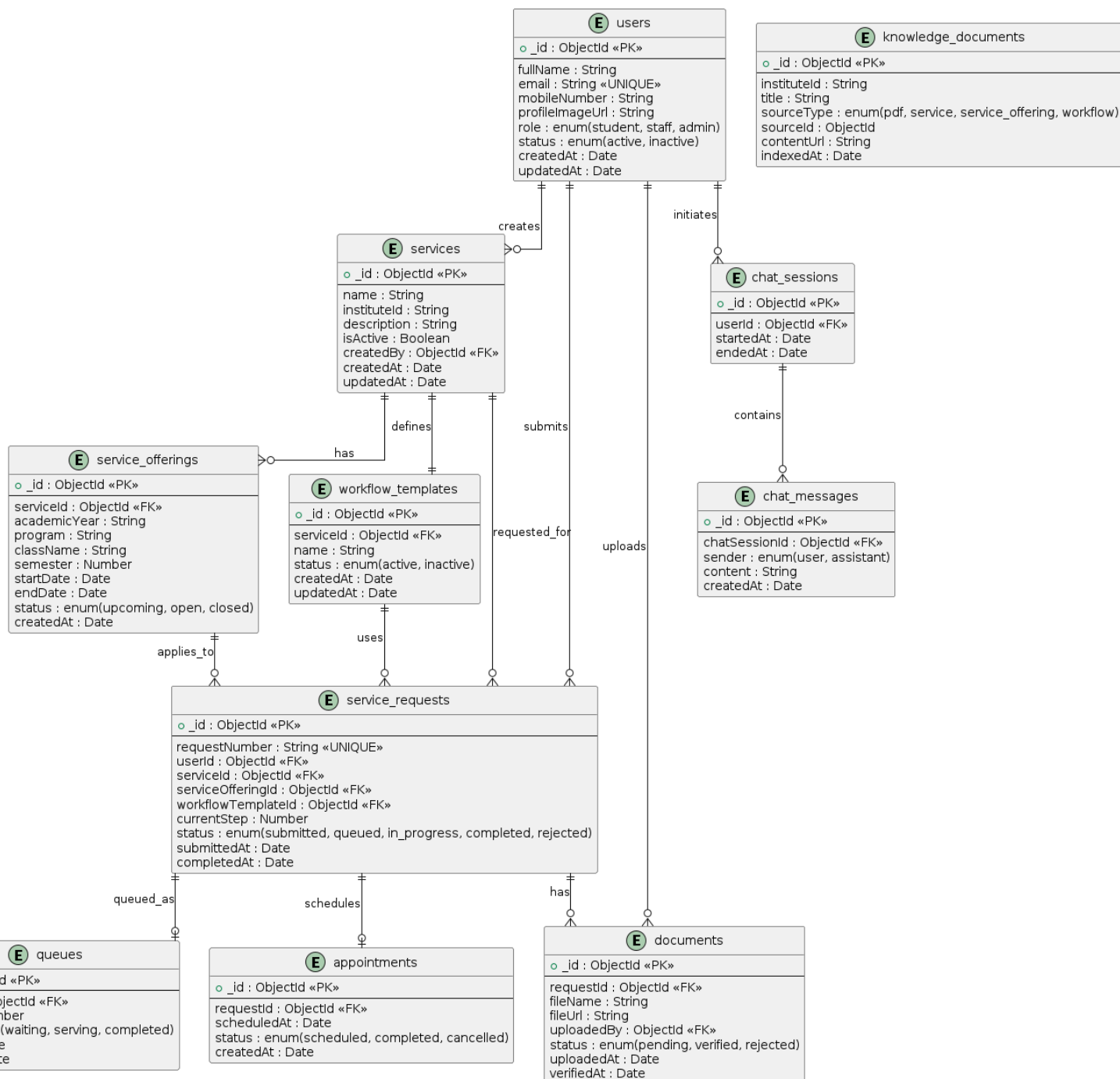
background workers using BullMQ.

Storage and Retrieval

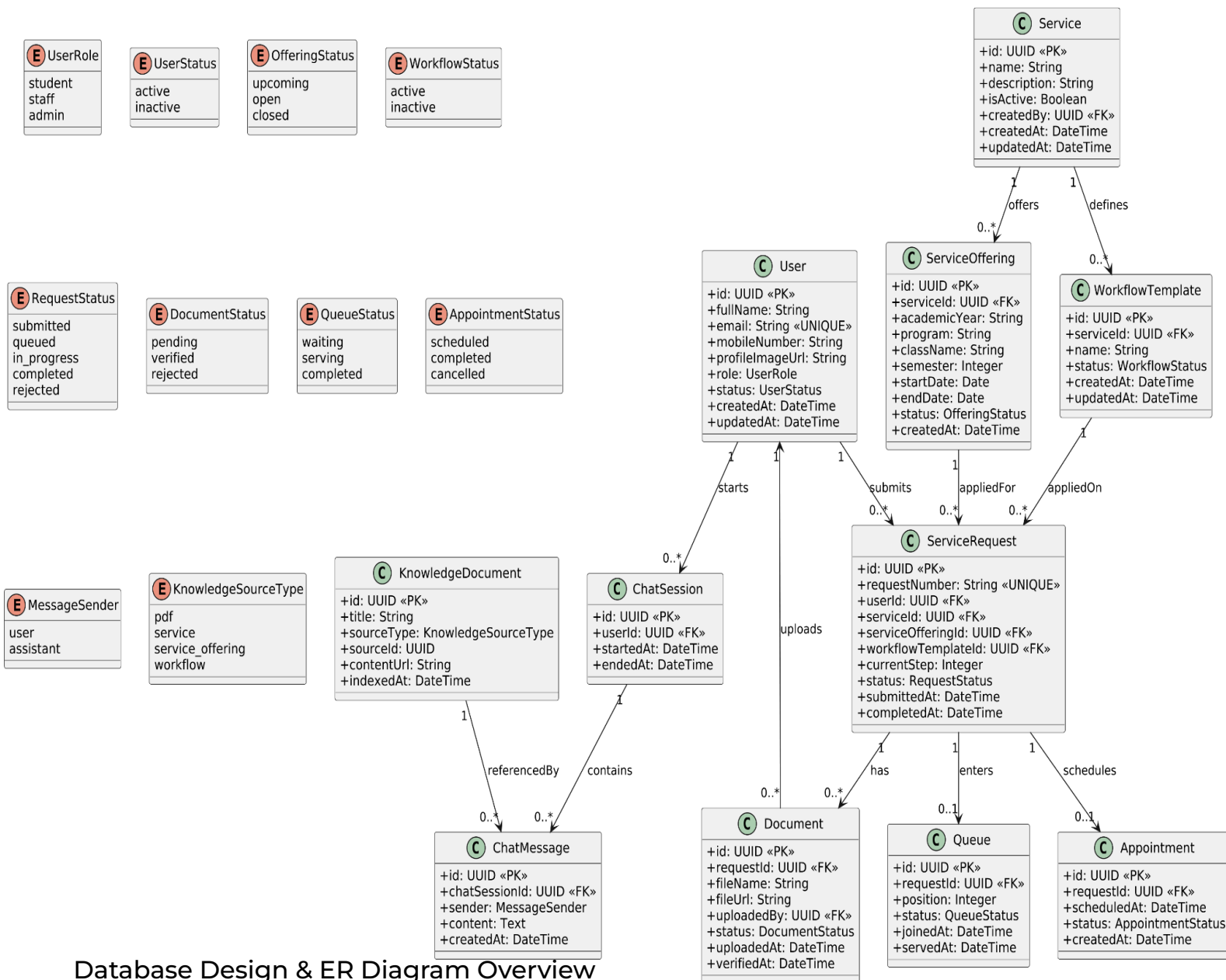
- **MongoDB** stores users, services, workflows, requests, documents, chats, queues, and appointments.
- **Redis** is used for caching and real-time queue state.
- **ImageKit** stores uploaded documents and images.
- A **knowledge index** is used for chatbot RAG responses.

5.4 Database Design

ER Diagram - Smart Workflow-Based Queue and Administrative Service Management System for Educational Institutions



COMPLETE DATABASE DESIGN - COLLEGE SERVICE MANAGEMENT SYSTEM



Database Design & ER Diagram Overview

6. Technology Stack and Justification

Frontend

- React**

React is used to build the frontend because it supports component-based development and helps in creating interactive and responsive web applications. It is suitable for building the student, staff, and admin portals.

- Zustand (State Management)**

Zustand is used for global state management because it is lightweight, simple to use, and works well for managing shared states like user sessions, queue status, and chatbot data.

- Tailwind CSS**

Tailwind CSS is used for styling because it allows fast UI development using utility classes and keeps styling consistent across the application.

- **ShadCN UI**
ShadCN UI is used for reusable UI components. It provides clean and accessible components that integrate well with Tailwind CSS.
- **Ant Design (Optional)**
Ant Design is used only where needed, mainly for complex tables and forms, as it provides ready-made components for such use cases.

Backend

- **Node.js**
Node.js is used as the backend runtime because it supports asynchronous processing and can handle multiple user requests efficiently.
- **Express.js**
Express.js is used as the backend framework to build REST APIs. It provides simple routing, middleware support, and easy integration with Node.js.
- **Monolithic Architecture**
The system uses a **monolithic backend architecture**, where all modules (authentication, workflow, requests, queue, chatbot) are part of a single application. This simplifies development, deployment, and maintenance for a college-scale system.
- **Stateful Authentication (Session-based)**
The system uses **stateful authentication** instead of JWT. User sessions are stored and managed on the server, making role-based access control easier to implement and manage.

Database

- **MongoDB**
MongoDB is used as the primary database because it supports flexible schemas and is suitable for storing workflow data, service requests, documents, and chat history.
- **Redis**
Redis is used for caching frequently accessed data, storing real-time queue states, and supporting background job processing

Other Tools and Frameworks

- **BullMQ**
BullMQ is used for background job processing such as queue handling, appointment scheduling, reminders, and escalation tasks. It is Redis-based and integrates well with the backend.
- **Vector Database (for Chatbot)**
A vector database is used to store embeddings for chatbot knowledge. This enables Retrieval-Augmented Generation (RAG) for better chatbot responses.
- **Chatbot API**
The chatbot uses a language model API to generate responses based on retrieved knowledge and user queries.

- **Deployment (Web Deployment)**

The application will be deployed as a web application. The hosting platform (such as Vercel or a VPS) will be decided later based on project requirements.

7. Proof of Concept (PoC)

Note: The Proof of Concept (PoC) will be implemented in a later phase of the project.

8. Testing and Validation Strategy

Unit testing approach

- **Scope:** Test service-layer logic (business rules), workflow step transitions, queue position calculations, document status changes, and worker job handlers.
- **Method:** Write small, fast unit tests that call service functions with mocked dependencies (mock DB, mock Redis, mock ImageKit, mock vector DB).
- **Goal:** Ensure each service method enforces rules (e.g., workflow step cannot advance unless documents verified) and workers call service methods (not DB models) as required.

Integration testing approach

- **Scope:** Verify end-to-end flows across modules: API → service layer → DB → cache → worker. Important flows: submit request → workflow instance → staff approve → complete; join queue → position updates; document upload → verification.
- **Method:** Use a test environment with real MongoDB and Redis instances (local or docker-compose). Run API endpoints (via Postman collection or test runner) against this environment to validate actual reads/writes and BullMQ job processing.
- **Goal:** Catch integration bugs (schema mismatches, wrong indexes, cache invalidation issues, worker-job wiring).

Manual and automated testing

- **Automated:** Unit tests + integration test suites run in CI for every pull request. Include automated checks for service contracts and basic export generation.
- **Manual:** Acceptance tests for UI flows (student submit, staff verify, queue view) and PoC demo script. Use the Postman collection for manual API checks and to record demo scenarios.
- **PoC validation:** Use sample data from PoC to run acceptance checklist (submit request, upload doc to ImageKit, run worker jobs, generate CSV export, chatbot lookup).

9. Risks, Challenges, and Mitigation

Risk: Technical complexity (workflow engine, SLA(Service Level Agreement), queue logic)

- **Problem:** Workflow rules, SLA escalation, and queue priority rules can be tricky and error-prone.
- **Mitigation:**
 - Start with a small, well-documented workflow template implementation and reuse it across services.
 - Implement step-by-step unit tests for each rule.
 - Simulate SLA escalation in the PoC and validate with worker jobs before full rollout.

Risk: Dependency on third-party tools (ImageKit, external model API/Gemini, vector DB)

- **Problem:** Outage or API quota changes can break document storage or chatbot features.
- **Mitigation:**
 - Add graceful fallback: for chatbot, fall back to deterministic responses if RAG fails; for ImageKit, allow temporary local storage and retry upload.
 - Add retries and exponential backoff for external calls.
 - Cache essential knowledge/doc metadata in Mongo/Redis to reduce immediate dependency.

Risk: Worker and queue reliability (BullMQ + Redis)

- **Problem:** Job failures or Redis outage can halt reminders, reordering, exports.
- **Mitigation:**
 - Ensure idempotent job handlers and job retry policies.
 - Use Redis persistence/backup and monitor queue lengths.
 - Provide degraded behavior (e.g., allow manual queue reassignments in staff UI) if workers are down.

Risk: Time & resource constraints

- **Problem:** Limited time to harden production features (notifications, scale tests).
- **Mitigation:**
 - Prioritize core flows first (submit → workflow → verification → export).

- Deliver incremental features using PoC as baseline; leave non-critical items (full audit formatting, advanced ranking for RAG) for Phase-3.

Risk: Security & session handling (stateful auth)

- **Problem:** Stateful sessions require secure session store, rotation, and protection against session fixation.
- **Mitigation:**
 - Store sessions in a secure, backed store (Redis or DB) with proper TTL.
 - Enforce HTTPS, same-site cookies, and session invalidation on logout.
 - Add rate limiting and input validation at API layer.

10. Phase 2 Outcomes and Readiness for Phase 3

What has been completed in Phase 2

- Detailed system design and diagrams (HLD, LLD, class UML, ER diagram, PlantUML dataflow).
- Module-wise design with responsibilities and service-layer contract (controllers → services → workers).
- Database design (MongoDB schema, Redis key usage) and ER diagrams.
- PoC implementing core end-to-end flow: stateful login, submit request, document upload (ImageKit metadata), workflow runtime, virtual queue + appointment, background jobs (BullMQ), basic chatbot pre-check + RAG lookup, CSV export.

System readiness for implementation

- **Ready parts:** Core models, service contracts, workflow runtime, queue/appointment logic, document metadata flow, and worker patterns are validated by PoC. These can be extended to full implementation.
- **Needs before production:** Security hardening (session store, rate limits), notification integrations (email), production deployment plan, production monitoring, and full audit/log formatting.

Key inputs carried forward to Phase 3

- LLD artifacts (UML, class diagrams, ER) and PlantUML source for developer reference.
- Service-layer skeletons and PoC code patterns (controllers, services, worker usage).
- Postman collection and demo scripts for acceptance tests.

- Seed knowledge documents and vector index snapshot for chatbot RAG tuning.
- Export templates (CSV spec) and example exports used for ERP sync design.
- Test cases and basic CI test suites to expand in Phase-3.